

Desarrollo Web Integrado

Semana 04: Implementación práctica de JPA y Spring Data en una arquitectura por capas

Mg. Ing. Enrique Lee Huamani Uriarte



**Universidad
Tecnológica
del Perú**

Dudas de la clase anterior

- ¿LES QUEDÓ CLARO CÓMO FUNCIONA @GETMAPPING Y EL USO DE @PATHVARIABLE Y @REQUESTPARAM?
- ¿PUDIERON EJECUTAR LOS ENDPOINTS EN POSTMAN O NAVEGADOR?

Saberes previos

- ¿Qué es una entidad en programación?
- ¿Conocen qué hace una base de datos relacional?

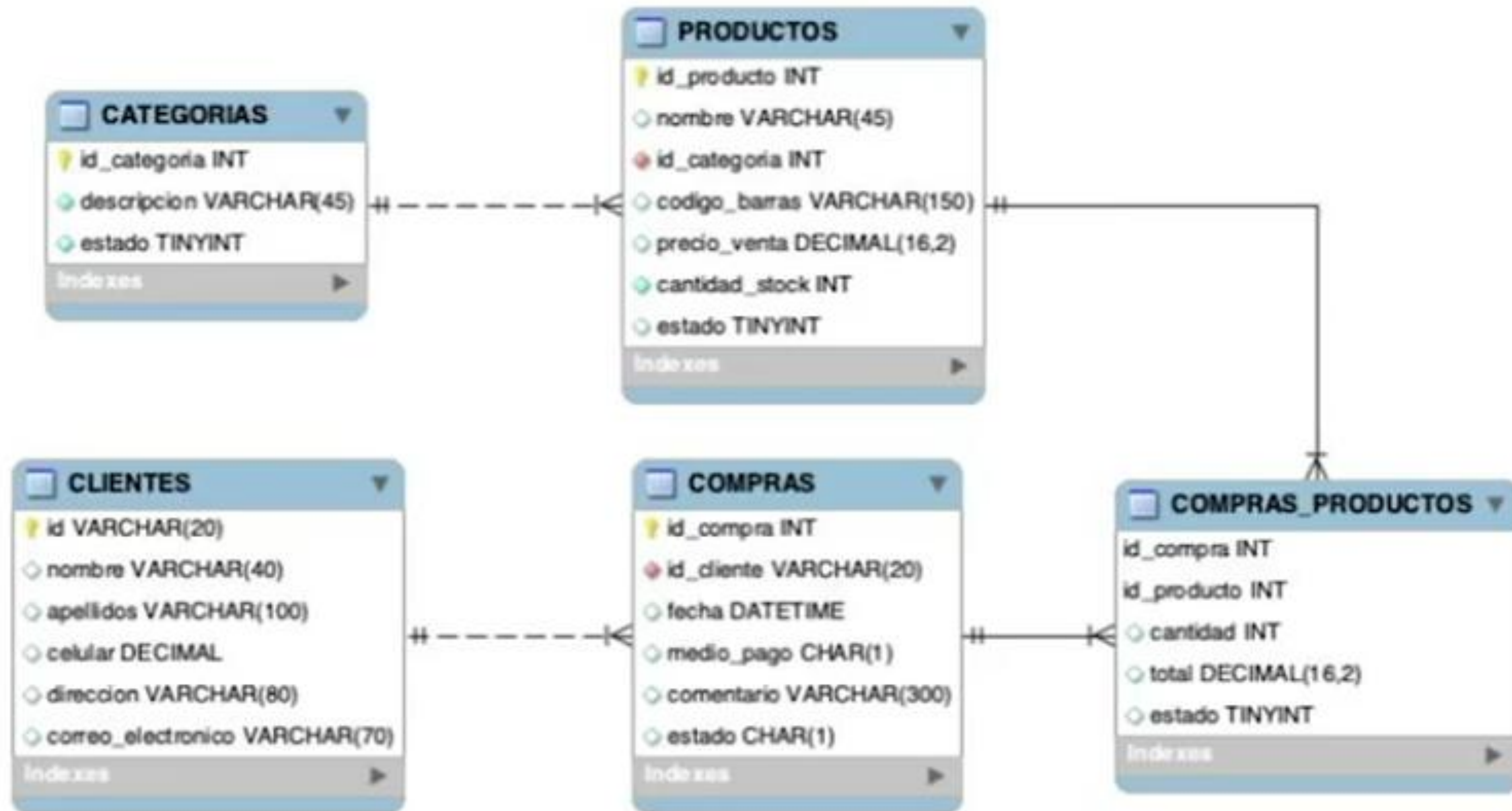
UTILIDAD

- ❑ Aprenderás a estructurar tu proyecto profesionalmente por capas.
- ❑ Entenderás cómo se guarda información en una base de datos usando JPA.
- ❑ Comenzarás a separar responsabilidades: controlador, servicio, repositorio, entidad

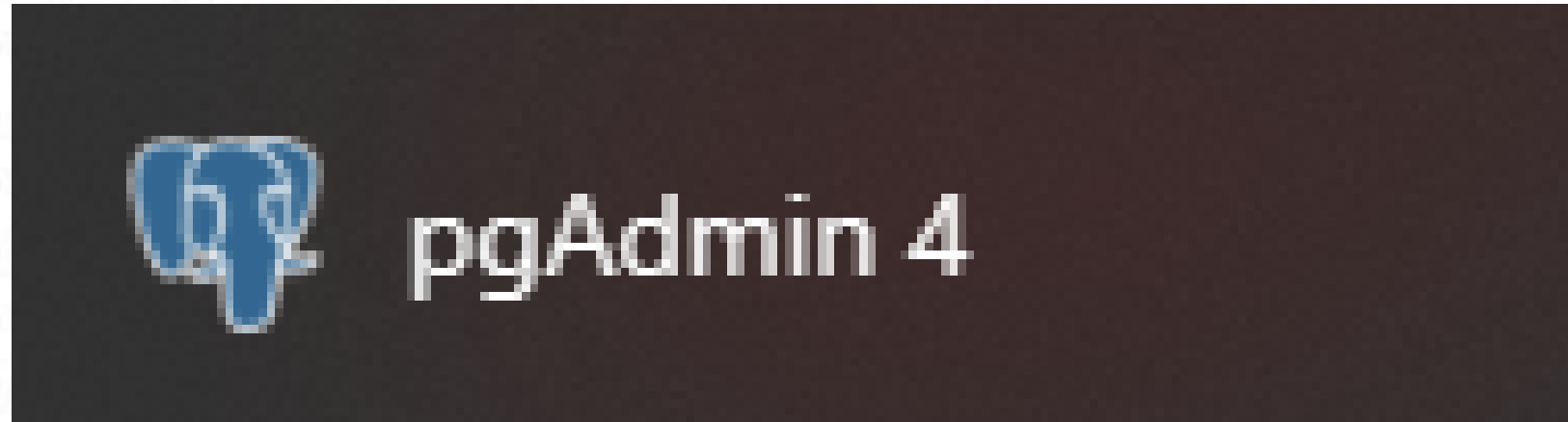
LOGRO DE LA SESIÓN

- ❑ Crear una estructura organizada en Spring Boot
- ❑ Definir entidades usando @Entity
- ❑ Usar JpaRepository para guardar información
- ❑ Conocer el flujo completo de una arquitectura por capas

MODELOS DE DATOS



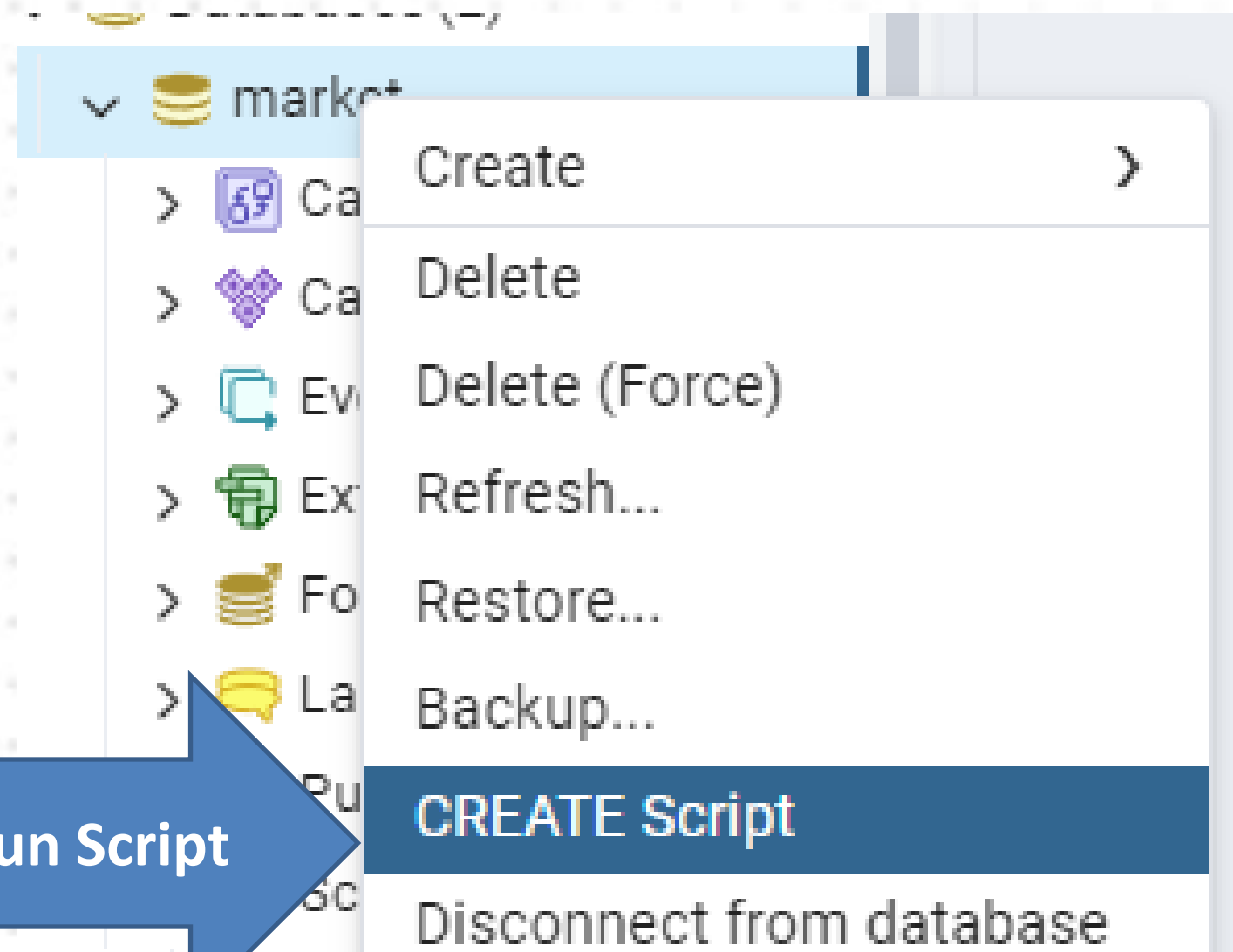
INICIAMOS NUESTRO GESTOR DE BASE DE DATOS



INTRODUCIMOS LA CONTRASEÑA RESPECTIVA CREAR PARA ESTE SERVICIO

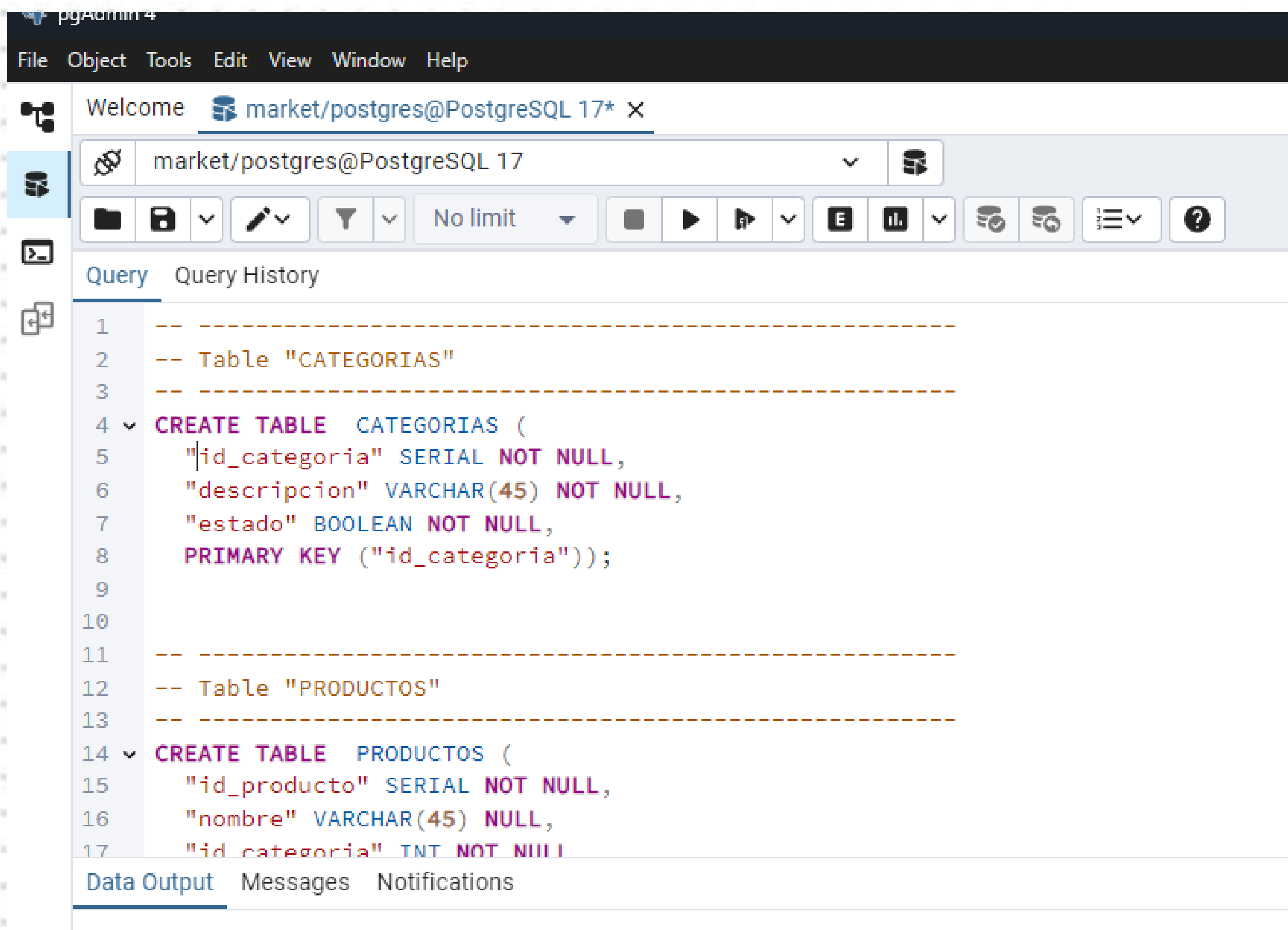
A screenshot of the "Connect to Server" dialog box in pgAdmin. The dialog has a title bar with a close button (X). The main text reads: "Please enter the password for the user 'postgres' to connect the server - 'PostgreSQL 17'". Below this is a large, empty text input field. Under the input field is a checkbox labeled "Save Password", which is currently unchecked. At the bottom right of the dialog are two buttons: "Cancel" with a close icon (X) and "OK" with a checkmark icon (✓).

CREAMOS UNA BASE DE DATOS CON NOMBRE market

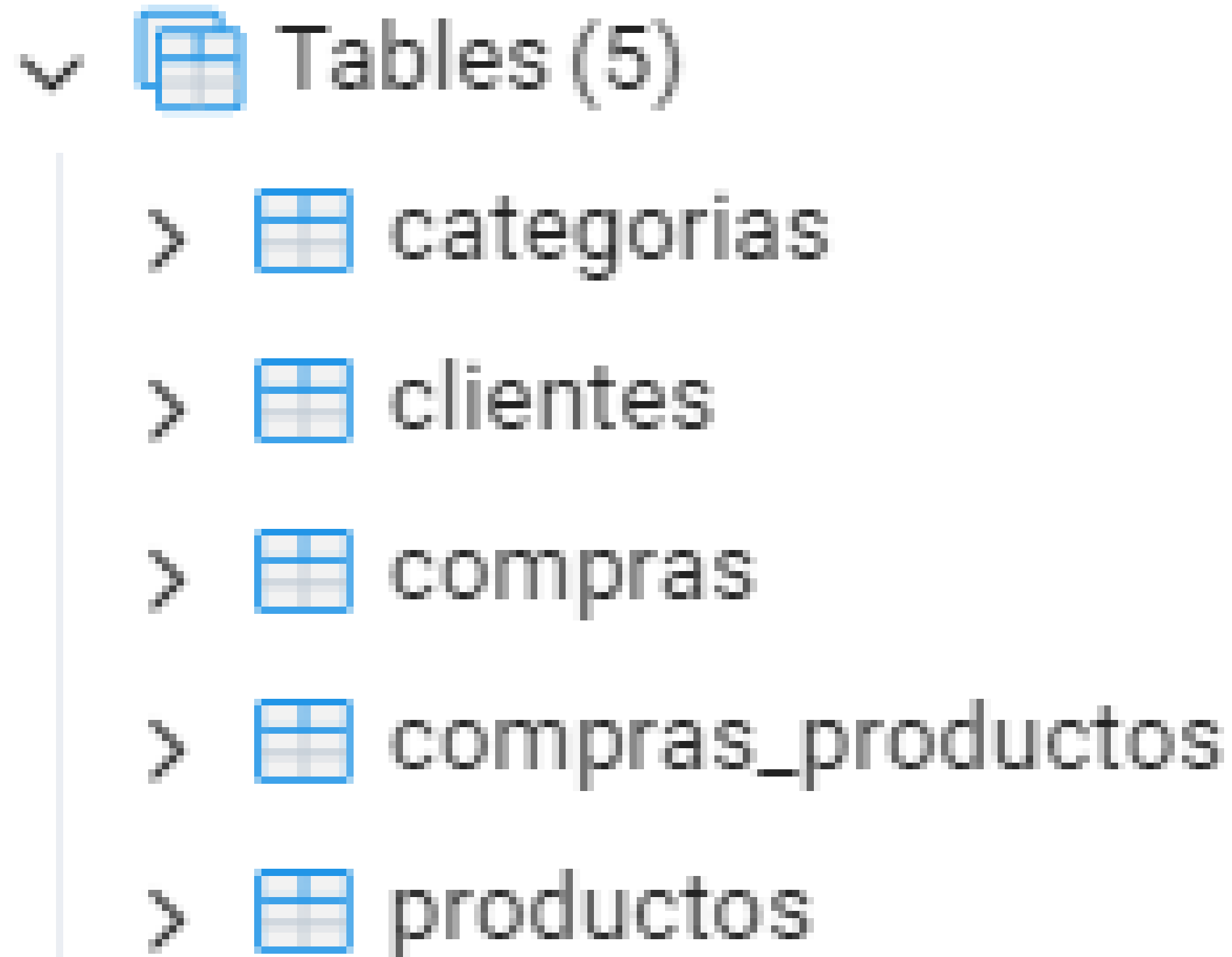
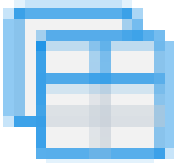
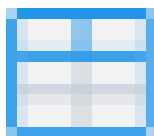
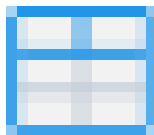
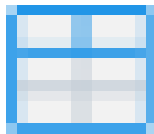
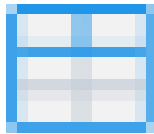
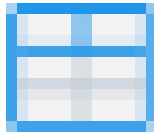


Crear un Script

EJECUTAMOS EL SCRIPT



PODEMOS VISUALIZAR QUE EXISTE LOS SIGUIENTES TABLAS CREADAS

- 
- A screenshot of a database management system interface. At the top, there is a light blue horizontal bar. Below it, a tree view shows a folder icon and the text 'Tables (5)'. To the left of this folder is a checkmark. Below the folder, there is a vertical line, and to its right, five table names are listed, each preceded by a right-pointing chevron and a small table icon. The table names are 'categorias', 'clientes', 'compras', 'compras_productos', and 'productos'.
- ✓  Tables (5)
 - >  categorias
 - >  clientes
 - >  compras
 - >  compras_productos
 - >  productos

EJECUTAR EL SET DE DATOS INICIALES

pgAdmin 4

File Object Tools Edit View Window Help

Welcome market/postgres@... x market/postgres@PostgreSQL 17* x

market/postgres@PostgreSQL 17

No limit

Query Query History

```
1  -- CATEGORIAS
2  INSERT INTO categorias VALUES (1, 'Frutas y verduras', true);
3  INSERT INTO categorias VALUES (2, 'Pastelería', true);
4  INSERT INTO categorias VALUES (3, 'Carne y pescados', true);
5  INSERT INTO categorias VALUES (4, 'Lácteos y huevos', true);
6  INSERT INTO categorias VALUES (5, 'Bebidas', true);
7  INSERT INTO categorias VALUES (6, 'Licores', true);
8  INSERT INTO categorias VALUES (7, 'Cuidado personal', true);
9  INSERT INTO categorias VALUES (8, 'Despensa', true);
10
11 -- PRODUCTOS
12 INSERT INTO productos VALUES (1, 'Guayaba Feijoa', 1, '7029 A42 23', 300, 500, true);
13 INSERT INTO productos VALUES (2, 'Mango', 1, '0316 R56 01', 2100, 250, true);
14 INSERT INTO productos VALUES (3, 'Manzana', 1, '7923 T23 19', 700, 130, true);
15 INSERT INTO productos VALUES (4, 'Aguacate', 1, '9322 Q33 02', 2500, 98, true);
16 INSERT INTO productos VALUES (5, 'Lechuga', 1, '9742 S22 21', 4000, 86, true);
17 INSERT INTO productos VALUES (6, 'Tomate', 1, '0483 R00 97', 290, 430, true);
```

Data Output Messages Notifications

Agregame una nueva dependencia – “postgresql”

Search

Found 1484 results

Sort by: Relevance ▼



1. PostgreSQL JDBC Driver

org.postgresql » postgresql

PostgreSQL JDBC Driver Postgresql

Last Release on Jan 14, 2025

5,409 usages

BSD

VERSION MAS MODERNA

Home » org.postgresql » postgresql



PostgreSQL JDBC Driver

PostgreSQL JDBC Driver Postgresql

License	BSD 2-clause
Categories	JDBC Drivers
Tags	database sql jdbc postgresql driver rdbms
HomePage	https://jdbc.postgresql.org
Ranking	#105 in MvnRepository (See Top Artifacts) #2 in JDBC Drivers
Used By	5,409 artifacts

Central (194)Atlassian 3rd-P Old (1)Redhat GA (19)Redhat EA (2)Brekka (1)HuaweiCloudSDK (1)OrbisGIS (1)ICM (3)

	Version	Vulnerabilities	Repository	Usages	Date
42.7.x	42.7.5		Central	407	Jan 14, 2025
	42.7.4		Central	552	Aug 22, 2024
	42.7.3		Central	613	Mar 14, 2024
	42.7.2		Central	367	Feb 20, 2024
	42.7.1	1 vulnerability	Central	281	Dec 06, 2023
	42.7.0	1 vulnerability	Central	95	Nov 20, 2023
	42.6.2		Central	118	Mar 13, 2024

Incluimos en el pom.xml

```
<!-- https://mvnrepository.com/artifact/org.postgresql/postgresql -->  
<dependency>  
  <groupId>org.postgresql</groupId>  
  <artifactId>postgresql</artifactId>  
  <version>42.7.5</version>  
  <scope>runtime</scope>  
</dependency>
```


Incluimos en el pom.xml

Ingresamos a la
primera propiedad

resources

static

templates

application.properties

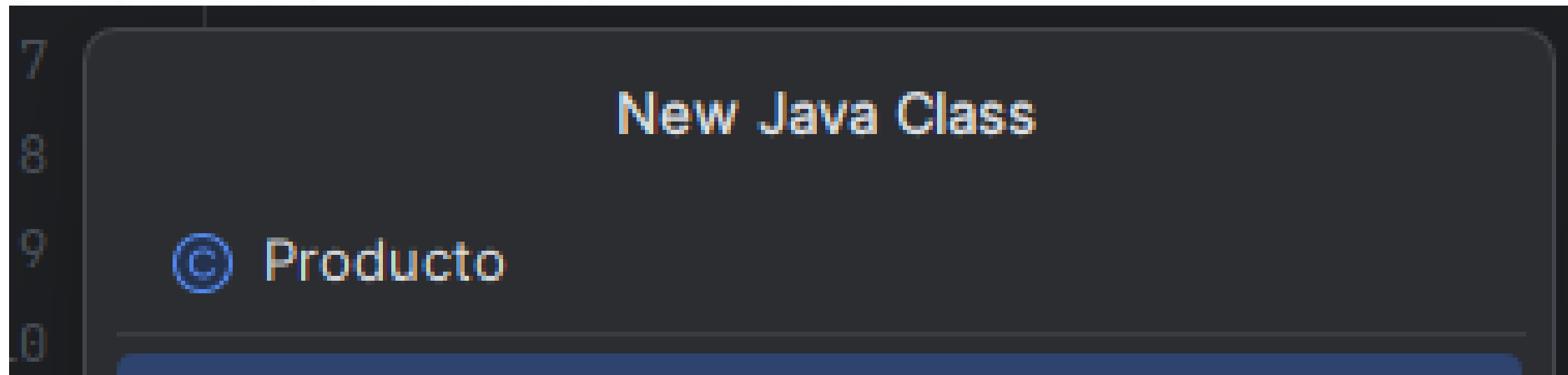
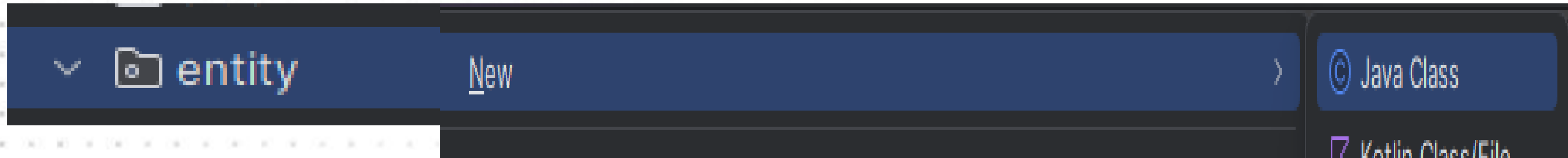
application-dev.properties

application-pdn.properties

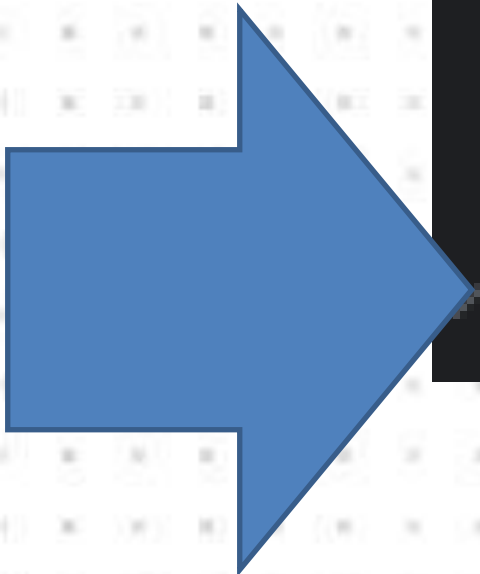
application.properties

```
spring.application.name=ProyectoModelo  
  
server.port=80  
# Database  
spring.datasource.url=jdbc:postgresql://localhost:5432/market  
spring.datasource.username=postgres  
spring.datasource.password=123456
```

MAPEAR LAS TABLAS COMO CLASES

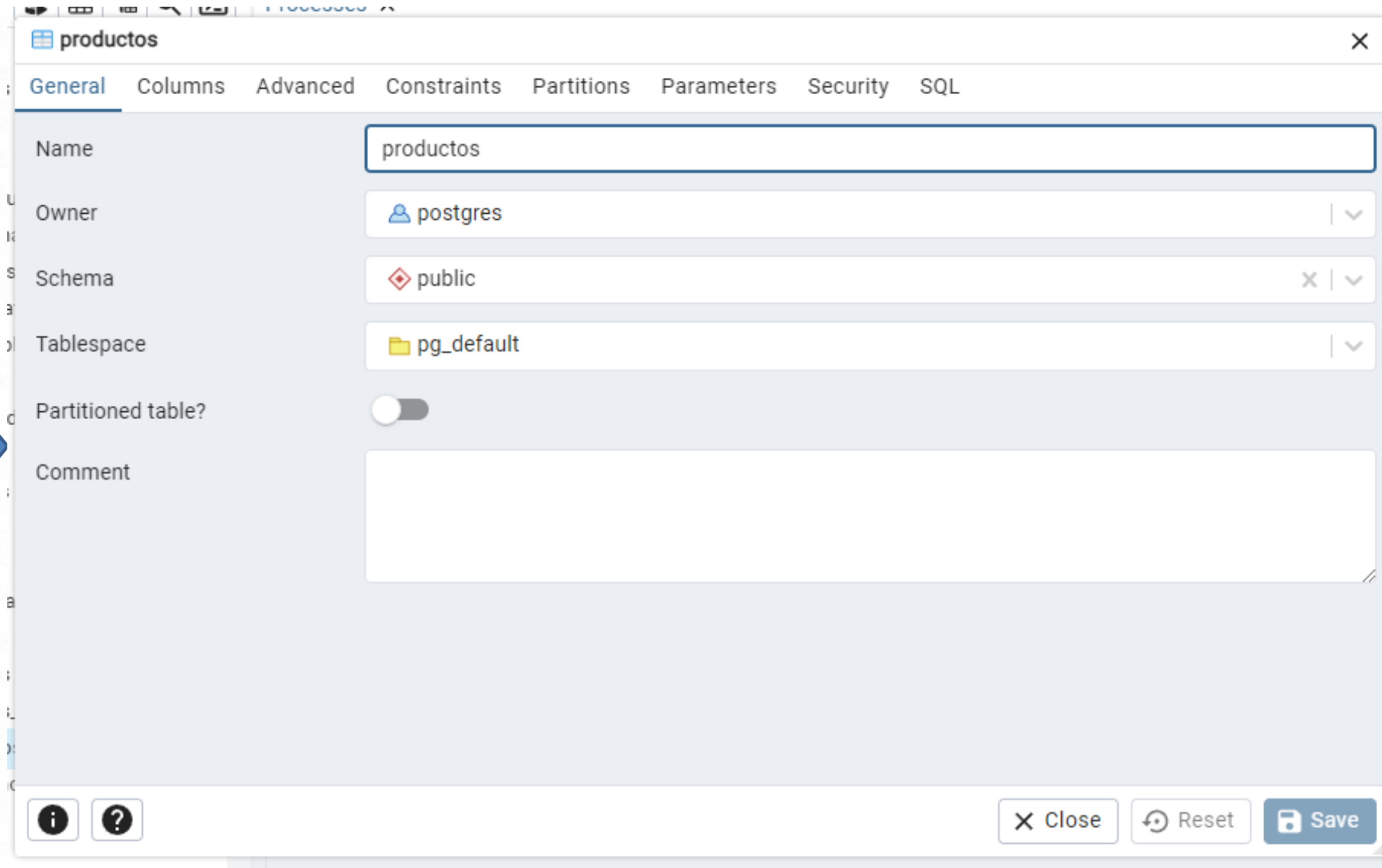


INGRESAMOS EL ATRIBUTO @Entity



```
package com.prueba.ProyectoModelo.persistence.entity;  
  
import jakarta.persistence.*;  
  
@Entity no usages
```

Es momento de crear nuestros atributos - primero verificamos las características de mi tabla Productos

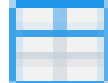
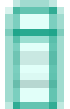


The screenshot shows a window titled 'productos' with a close button (X) in the top right corner. The window has several tabs: 'General', 'Columns', 'Advanced', 'Constraints', 'Partitions', 'Parameters', 'Security', and 'SQL'. The 'General' tab is selected and highlighted. It contains the following fields:

- Name:** A text box containing 'productos'.
- Owner:** A dropdown menu showing 'postgres' with a user icon.
- Schema:** A dropdown menu showing 'public' with a diamond icon.
- Tablespace:** A dropdown menu showing 'pg_default' with a folder icon.
- Partitioned table?:** A toggle switch that is currently turned off.
- Comment:** A large text area for adding a comment.

At the bottom of the window, there are three buttons: 'Close' (with an X icon), 'Reset' (with a circular arrow icon), and 'Save' (with a floppy disk icon). On the left side of the window, there are two small icons: an information icon (i) and a help icon (?).

TAMBIEN VEREMOS LAS CARACTERISTICAS DE SUS VARIABLES

▼  productos
▼  Columns (7)
 id_producto
 nombre
 id_categoria
 codigo_barras
 precio_venta
 cantidad_stock
 estado

Los que realizamos en este apartado es darle el formato de buenas practicamos para poder utilizar en del desarrollo

```
@Table(name = "productos")  
  
public class Producto {
```

TAMBIEN EN EL CASO DE ASIGNAR EL ID INCREMENTAL

```
@Id 2 usages  
@GeneratedValue(strategy = GenerationType.IDENTITY)  
@Column(name = "id_producto")  
private String idProducto;
```

Como también el cambio de los nombre de las variables

```
@Column(name = "id_producto")  
private String idProducto;  
  
-----  
@Column(name = "id_producto")  
private String idProducto;
```

**SIN NINGUNA COMPLICACION EN EL CASO NO SEA
NECESARIO CAMBIAR**

```
private String nombre; 2 usages
```



Universidad
Tecnológica
del Perú

SEGUIDAMENTE DE MUESTRO TODO EL CODIGO DE PRODUCTOS

```
package com.prueba.ProyectoModelo.persistence.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_producto")
    private String idProducto;

    private String nombre;

    @Column(name = "id_categoria")
    private Integer idCategoria;

    @Column(name = "codigo_barras")
    private String codigoBarras;

    @Column(name = "precio_venta")
    private Double precioVenta;

    @Column(name = "cantidad_stock")
    private Integer cantidadStock;
```

```
private Boolean estado;

    public String getIdProducto() {
        return idProducto;
    }

    public void setIdProducto(String idProducto) {
        this.idProducto = idProducto;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public Integer getIdCategoria() {
        return idCategoria;
    }

    public void setIdCategoria(Integer idCategoria) {
        this.idCategoria = idCategoria;
    }
```

SEGUIDAMENTE DE MUESTRO TODO EL CODIGO DE PRODUCTOS

```
public String getCodigoBarras() { no usages
    return codigoBarras;
}

public void setCodigoBarras(String codigoBarras) { no usages
    this.codigoBarras = codigoBarras;
}

public Double getPrecioVenta() { no usages
    return precioVenta;
}

public void setPrecioVenta(Double precioVenta) { no usages
    this.precioVenta = precioVenta;
}

public Integer getCantidadStock() { no usages
    return cantidadStock;
}

public void setCantidadStock(Integer cantidadStock) { no usages
    this.cantidadStock = cantidadStock;
}
```

```
public Boolean getEstado() { no usages
    return estado;
}

public void setEstado(Boolean estado) { no usages
    this.estado = estado;
}
}
```


EJERCICIO

REALIZAR LO MISMO PARA

- **PRODUCTOS**
- **CATEGORIA**
- **CLIENTE (UN TIC #EL CLIENTE YA TIENE SU LLAVE PRIMARIA DEFINIDA#)**

¿Dudas consultas?



Cierre:

- La arquitectura usada en este proyecto es **por capas y orientada al dominio**.
- Hoy aprendiste:
 - Cómo se estructura un proyecto profesional
 - Qué es JPA y cómo se conecta con la base de datos Cómo fluye la información desde el cliente hasta la base de datos y de regreso
 - Prepárate para la próxima clase: realizaremos un CRUD básico



**Universidad
Tecnológica
del Perú**

